

Engage360

🕒 Created	@November 25, 2025 7:25 AM
🏷️ Tags	

Technical Plan (Flutter + Firebase)

Engage360: Volunteer Mobile Application

1. Introduction

This technical plan outlines the technologies, architecture, development tools, and implementation approach used for building Engage360 using **Flutter** and **Firebase Free Tier**. The objective is to ensure a **100% zero-cost development environment** while maintaining proper mobile app standards such as real-time features, push notifications, cloud database, and gamification.

Flutter is chosen for cross-platform mobile development, while Firebase provides a free, serverless backend solution suitable for authentication, real-time data, storage, and notifications.

2. Technology Stack Overview

2.1 Frontend Technology

Technology	Purpose	Cost
Flutter (Dart)	Mobile app development (Android/iOS)	Free
Figma	UI/UX design	Free tier
GitHub	Version control	Free

2.2 Backend Technology (Firebase — Free Tier)

Firestore Service	Usage	Free Tier Capability
Firestore Authentication	Email/Password login	Free
Firestore Cloud Firestore	Cloud NoSQL database	50k reads/day free
Firestore Storage	Images, certificates	5GB free
Firestore Cloud Functions	Business logic triggers	Background triggers free
Firestore Cloud Messaging	Push notifications	Free
Firestore Analytics	User behavior	Free
Firestore Emulator Suite	Local testing	Free

The Firebase Spark Plan (free plan) fully supports this project and does **not** require any payment.

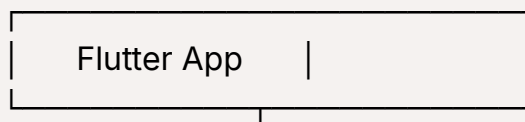
3. System Architecture

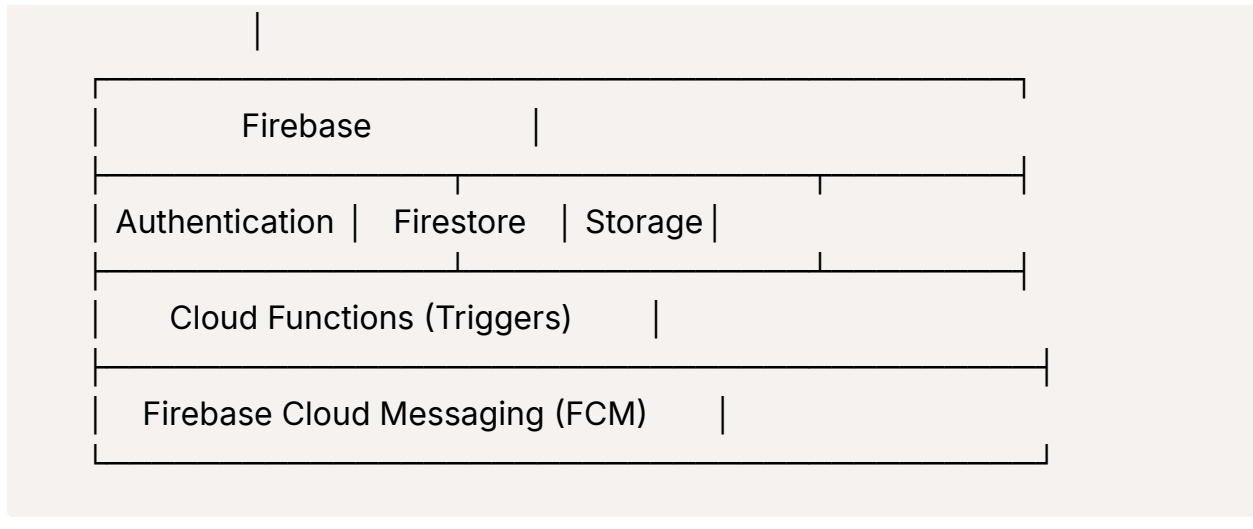
The system follows a **client-serverless architecture**, where the Flutter app directly interacts with Firebase services.

Architecture Components

- Flutter App (Client)
- Firestore Authentication
- Firestore Cloud Firestore
- Firestore Storage
- Firestore Cloud Functions (automatic triggers)
- Firestore Cloud Messaging (Push Notifications)

Architecture Diagram (Text Form)





4. System Modules & Firebase Integration

Below is the full mapping of modules to Firebase services.

4.1 User Authentication Module

- Email/Password authentication (Firebase Auth)
- User profile stored in Firestore
- Profile images stored in Firebase Storage

4.2 Volunteer Program Module

- Activities stored in `activities` collection
- Image posters stored in Firebase Storage
- CRUD operations through Firestore Security Rules

4.3 Registration & Volunteer Tracking Module

- Registrations stored in `registrations` collection
- Cloud Functions trigger:
 - Assign points
 - Update status

- Mark completion

4.4 Gamification & Points Module

- Points field stored in the `users` collection
- Cloud Function adds points when:
 - User joins activity
 - User completes activity

4.5 Certificate Module

Generate PDF in Flutter

- No Cloud Functions needed
- PDF saved locally or uploaded to Firebase Storage

4.6 Messaging Module

- Stored in `messages` collection
- Real-time updates via Firestore snapshot listeners
- Push notifications sent through Cloud Functions + FCM

4.7 Notification Module

- New activity
- New message
- Activity reminders

All created via **FCM**, which is free.

5. Firebase Database Structure

Collections

users/
organizations/
activities/
registrations/
messages/
certificates/
notifications/

Example Document Structure

users

```
{  
  name: "",  
  email: "",  
  role: "volunteer" | "organization",  
  phone: "",  
  points: 0,  
  profilePhotoUrl: "",  
  joinedActivities: []  
}
```

activities

```
{  
  title: "",  
  description: "",  
  location: "",  
  startDate: "",  
  endDate: "",  
  organizationId: "",  
  posterUrl: ""  
}
```

messages

```
{
  senderId: "",
  receiverId: "",
  content: "",
  timestamp: ""
}
```

6. Cloud Functions Usage

All functions will be **background-triggered** to avoid paid HTTP functions.

Functions Implemented

1. onUserRegistersForActivity

→ Add points, send notification

2. onMessageCreated

→ Trigger FCM push notification

3. onActivityCompleted

→ Auto-generate certificate (optional)

4. Daily Reminder Function (Free Workaround)

- Triggered by a Firestore update, not scheduled cron (Spark Plan restriction)

7. Firebase Security Rules

Authentication

```
allow read, write: if request.auth != null;
```

User Profile

```
allow update: if request.auth.uid == userId;
```

Organizations Managing Activities

```
allow create, update, delete: if resource.data.orgId == request.auth.uid;
```

Messages

```
allow read, write:  
if request.auth.uid == senderId  
|| request.auth.uid == receiverId;
```

These rules keep the project secure without needing any paid services.

8. Flutter Development Plan

8.1 Folder Structure

```
lib/  
├── features/  
│   ├── auth/  
│   ├── profile/  
│   ├── activities/  
│   ├── messaging/  
│   ├── certificates/  
│   └── rewards/  
└── services/  
    ├── firebase_auth_service.dart  
    ├── firestore_service.dart  
    └── storage_service.dart
```

```
| | — notification_service.dart
|
| — utils/
| — main.dart
```

9. Testing Approach

Tools

- Firebase Emulator (free)
- Flutter test
- Manual UAT on Android phone (free)

Tests

- Authentication flow
- Activity creation/joining
- Firestore read/write rules
- Push notification delivery
- Messaging flow
- Points calculation

10. Deployment Plan

Android

- ✓ Build APK / AAB
- ✓ Share directly with supervisor
- ✓ No Google Play cost required

ios

✗ Publishing requires Apple Developer fee

11. Conclusion

This technical plan provides a functional volunteer management mobile application. Flutter and Firebase's free tier allow real-time functionality, secure login, gamification, messaging, and notifications without incurring any cost.